

AnoGAN i wykrywanie fake newsów z wykorzystaniem LLM

Przedmowa:

Pomysł na ten projekt zrodził się podczas rozmowy z dr Pawlickim na temat sieci neuronowych. Rozmawialiśmy o tym, jak komputer uczy się rozpoznawać obiekty za pomocą sieci neuronowych, a on uświadomił mi, że komputer „zaakceptuje” wszystko, o ile będzie miało logiczną reprezentację numeryczną.

W przypadku obrazów składają się one z pikseli, a piksele są po prostu liczbami. Komputer nie przejmie się tym, co reprezentują liczby; zawsze będzie próbował znaleźć w nich wzorce. To doprowadziło mnie do pytania: jeśli sieci oparte na GAN, w tym AnoGAN, są zaprojektowane przede wszystkim do analizy obrazów, dlaczego nie można ich również używać do analizy sekwencji zdań i porównywania ich ze sobą?

Ten eksperyment ma na celu zbadanie, czy możliwe jest dostosowanie AnoGAN do analizy tekstu, w szczególności do wykrywania fałszywych wiadomości. Hipoteza jest taka, że fałszywe wiadomości można traktować jako anomalię w przestrzeni prawdziwych artykułów informacyjnych, tak jak AnoGAN wykrywa anomalie w obrazach.

Aby to udowodnić:

1. Konwertuję dane tekstowe na reprezentacje numeryczne przy użyciu kodowania par bajtów (BPE) i osadzeń.
2. Trenuję AnoGAN na prawdziwych wiadomościach, aby nauczył się generować autentyczne wzorce tekstowe.
3. Używam błędu rekonstrukcji jako metryki wykrywania anomalii, sprawdzając, czy fałszywe artykuły informacyjne znacząco odbiegają od prawdziwych.

Celem jest ustalenie, czy niesupervised podejście, takie jak AnoGAN, pierwotnie zaprojektowane dla obrazów, może skutecznie wykrywać fałszywe wiadomości na podstawie ich struktury tekstowej. Jeśli się powiedzie, może to otworzyć drzwi do wykorzystania generatywnych modeli wykrywania anomalii w przetwarzaniu języka naturalnego (NLP) i innych zadaniach opartych na tekście.

Architektura AnoGAN i proces trenowania/testowania

Model **AnoGAN** składa się z dwóch konkurujących sieci: generatora i dyskryminatora, zwykle z głębokimi warstwami konwolucyjnymi (architektura DCGAN). Generator uczy się tworzyć przykłady „normalne” (np. realistyczne obrazy) z wektora latentnego, a dyskryminator ocenia, czy próbka jest rzeczywista czy wygenerowana. W fazie trenowania obie sieci grają w przeciwny proces, gdzie dyskryminator maksymalizuje poprawną

klasyfikację danych rzeczywistych vs. sztucznych, a generator uczy się *oszukać* dyskryminator poprzez minimalizację jego straty. Po zakończeniu treningu generator nauczył się rozkładu danych normalnych.

W fazie testowej dla nowego wejścia x AnoGAN poszukuje wektora z w przestrzeni latentnej, którego generowane wyjście $G(z)$ będzie jak najbardziej zbliżone do x . W tym celu definiuje się funkcję straty z dwóch składników: błąd rekonstrukcji („residual loss” $L_R = \|x - G(z)\|$) oraz miarę odchylenia cech w reprezentacji dyskryminatora („discrimination loss” $L_D = \|f(x) - f(G(z))\|$). Parametry z są dostosowywane iteracyjnie przez propagację wsteczną, aby minimalizować łączną stratę $L = (1 - \lambda)L_R + \lambda L_D$. Uzyskany w ten sposób „anomaly score” jest miarą różnicy x od rozkładu normalnego – duża wartość (nieudana rekonstrukcja) oznacza anomalię.

AnoGAN w detekcji anomalii i kontekst tekstowy

Wykrywanie anomalii polega na identyfikacji danych znacząco odbiegających od „normalnej” dystrybucji. W kontekście tekstowym anomalie mogą obejmować treści o **złośliwym charakterze** – np. mowa nienawiści, cyberprzemoc czy dezinformacja. AnoGAN w oryginale działał na obrazach medycznych, lecz koncepcja przenoszenia jej na dane tekstowe wymaga przetłumaczenia dyskretnego tekstu na przestrzeń ciągłą. Na przykład, jako metrykę odległości między tekstami wykorzystuje się odległość między ich osadzonymi wektorami (embeddingami). Jednocześnie sieciowe generowanie sekwencji słów wymaga specjalnych technik: proponowano łączenie wyrazów w „paczki tekstowe” umożliwiające propagację gradientu i działanie dyskryminatora.

Definicja anomalii w sekwencjach tekstowych często sprowadza się do tego, że przykład jest „niestandardowy” w odniesieniu do zbioru danych treningowych. Na przykład Nyarko et al. definiują anomalię jako punkt danych **znacznie odbiegający** od normalnego rozkładu, zakładając, że tylko „pozytywne” (normalne) przykłady są znane, a anomalie są rzadkie i nieznane. W praktyce więc podejścia oparte na AnoGAN dla tekstu starają się rekonstruować typowe zdania i mierzyć rozbieżność nowych wiadomości od tego modelu językowego.

Badania i projekty: AnoGAN oraz warianty w wykrywaniu dezinformacji

Bezpośrednie zastosowanie AnoGAN do wykrywania fake newsów jest w literaturze rzadko opisywane, ale pojawiły się próby użycia GAN-ów do wykrywania nieprawdziwych treści. Na przykład ARAE-AnoGAN (2020) to metoda tekstowej detekcji anomalii, która wykorzystuje **adwersarialnie regulowany autoenkoder** (ARAE) do rekonstrukcji normalnych zdań i definiuje wynik anomalii na podstawie błędów rekonstrukcji i cech enkodera. Autor pracy pokazuje, że ARAE-AnoGAN efektywnie wykrywa nietypowe zdania jako anomalie.

Inny przykład, choć w dziedzinie recenzji (niekoniecznie fake news), to model **FakeGAN** (2018) zaproponowany do wykrywania fałszywych opinii. Używa on dwóch dyskryminatorów

i generatora trenowanego w trybie wzmacniania (RL), aby odróżnić recenzje szczere od fałszywych. Choć FakeGAN nie jest metodą typu AnoGAN, ilustruje próbę wykorzystania struktury GAN-ów w zadaniu klasyfikacji tekstu z fałszywymi informacjami. Istotnie, wyniki FakeGAN są porównywalne z najlepszymi klasyfikatorami typu supervised na zbiorze recenzji podróbek.

Wśród wariantów AnoGAN na obrazach warto wspomnieć o **f-AnoGAN** (2019), który przyspiesza fazę testową poprzez wytrenowanie dodatkowego enkodera do mapowania obrazów do przestrzeni latentnej. W f-AnoGAN proces odwrotnej projekcji nie wymaga iteracyjnej optymalizacji, a do wykrywania anomalii używa się łączonego błędu rekonstrukcji i błędu cech z dyskryminatora. Inny pokrewny model, **GANomaly** (2018), łączy autoenkoder z GAN-em: encoder–decoder–encoder pozwala uczyć się mapowania obrazu na wektor latentny i odwrotnie, a większe odległości w przestrzeni latentnej przy nieznanych (anomalii) przykładach wskazują na odchylenie od normy. Choć GANomaly i f-AnoGAN powstały dla obrazów, pokazują rozwijane podejścia zwiększające efektywność i dokładność detekcji anomalii.

Połączenia AnoGAN z modelami LLM w wykrywaniu fake newsów

Nowe **duże modele językowe (LLM)**, takie jak BERT czy GPT, stanowią źródło bogatych reprezentacji semantycznych tekstu. Mogą one wspomagać podejścia oparte na AnoGAN na różne sposoby. Po pierwsze, LLM mogą dostarczać wektorów cech (embeddingów) dla zdań – trenowanie AnoGANa w przestrzeni tych wektorów pozwoliłoby wychwycić anomalie semantyczne. Po drugie, LLM same w sobie oceniają wiarygodność tekstu; okazało się, że *fałszywe twierdzenia* mają wyższy „perplexity” (mniejsze prawdopodobieństwo) według modelu językowego niż prawdziwe informacje. Kang i in. demonstrują, że jeśli LLM jest najpierw „nakarmiony” wiarygodnymi dowodami, to zdebunkowanie fałszu można traktować jako zadanie minimalizacji perplexity: zdania niezgodne z prawdą są trudniejsze do poprawnego „przypasowania” przez model, co objawia się większym błędem.

Ograniczenia i kierunki przyszłych badań

Podejście AnoGAN do fake newsów ma jednak wiele wyzwań. Po pierwsze, oryginalny AnoGAN przeznaczony jest dla danych ciągłych (obrazy) – generowanie realistycznych sekwencji słów GAN-ami jest trudne z powodu dyskretnej natury tekstu, co utrudnia propagację gradientu. Zastosowanie hybryd (np. RNN jako generator) pomaga, ale nadal brakuje sprawdzonych mechanizmów do trenowania modeli GAN w pełni na tekstach. Po drugie, fenomen fake news często związany jest z subtelnymi faktograficznymi błędami czy manipulacjami kontekstem, które mogą nie manifestować się jako odchylenie w „statystycznej” warstwie języka. Modele oparte wyłącznie na anomaliach semantycznych mogłyby przeoczyć bardzo przekonujące fake newsy napisane w prawidłowym stylu językowym. Po trzecie, AnoGAN wymaga dużego zbioru „czystych” (uczciwych) danych do

nauki – w przypadku treści newsowych kosztowne jest ręczne zapewnienie, że zbiór treningowy nie zawiera żadnej dezinformacji.

Przyszłe prace mogą iść kilkoma ścieżkami. Integracja AnoGAN z LLM otwiera nowe możliwości: np. wykorzystanie transformatorów jako generatora tekstu (zamiast konwolucji) oraz jako ekstraktora cech dla dyskryminatora. Połączenie mechanizmów fakt-checkingu (np. weryfikacja zewnętrznych baz wiedzy) z anomalijnymi wskaźnikami językowymi może poprawić trafność detekcji. Ponadto, szybkie wersje AnoGAN (jak f-AnoGAN) redukują czas inferencji dzięki trenowaniu dodatkowego enkodera – podobne przyspieszenia warto zaimplementować w tekstowych adaptacjach. Wreszcie, wskazówką są prace nad detekcją semantycznych anomalii z użyciem LLM: mogą one sugerować, jak mierzyć niespójność logiczną lub faktograficzną w newsach. W sumie, hybrydyczne systemy łączące nienadzorowane uczenie anomalii (GAN) z nadzorowanymi modelami językowymi oraz ekspercką wiedzą wydają się najbardziej obiecującym kierunkiem dalszych badań.

Implementacja ANOGAN z LSTM + kod

Komórka 1: Importy

```
import numpy as np

import pandas as pd

import torch

import torch.nn as nn

import torch.optim as optim

import matplotlib.pyplot as plt

from torch.utils.data import DataLoader, TensorDataset

import seaborn as sns

np.random.seed(42)

torch.manual_seed(42)
```

Manual seed pozwala na zatrzymanie losowości, ułatwia to porównywanie i ewaluację modelu z poprzednimi wynikami.

Komórka 2: Implementacja LSTM i mechanizmu Attention w NumPy

1. Inicjalizacja wag i biasów czterech bramek

- W konstruktorze `__init__` stworzyłem macierze `Wf`, `Wi`, `Wc`, `Wo` o wymiarach $(\text{hidden_size} \times (\text{input_size} + \text{hidden_size}))$ i wektory przesunięć `bf`, `bi`, `bc`, `bo` o wymiarach $(\text{hidden_size} \times 1)$.
- Dzięki temu każda bramka LSTM (forget, input, candidate, output) ma własny zestaw parametrów, co wiernie odtwarza oryginalny wzór sieci.

2. Metoda `forward` – pojedynczy krok czasowy

- Połączyłem poprzedni stan ukryty `h_prev` i bieżące wejście `x` przez konkatencję pionową (`np.vstack`), co odpowiada $[h_{t-1}, x_t]$.
- Obliczyłem wskaźniki „forget” (`ft`), „input” (`it`), kandydat na nową pamięć (`c_tilde`) i „output” (`ot`) przy pomocy sigmoidy oraz tanh.
- Zaktualizowałem stan komórki `ct = ft * c_prev + it * c_tilde` i stan ukryty `ht = ot * tanh(ct)`.
- **Dlaczego to kluczowe?** Ręczna implementacja pokazuje, jak poszczególne bramki kontrolują przepływ informacji i utrzymywanie kontekstu w sekwencjach.

3. Metoda `forward_sequence` – przetwarzanie całej sekwencji

- Przeszedłem przez każdy wektor `x_t` w sekwencji `x_seq` (o kształcie $(\text{seq_len}, \text{input_size})$), wywołując `forward` i zbierając kolejne stany ukryte do listy.
- Na wyjściu zwróciłem macierz $(\text{hidden_size} \times \text{seq_len})$ z wszystkimi stanami oraz ostatnie `h`, `c`.
- **Dlaczego?** Potrzebuję pełnego przebiegu sekwencji zarówno do wizualizacji wyników, jak i do mechanizmu uwagi.

4. Rozszerzenie: `LSTMWithAttention`

- W konstruktorze dodałem niewielką macierz `W_a` ($1 \times \text{hidden_size}$) i bias `b_a` (1×1) dla score’u uwagi.
- W `forward_with_attention` najpierw wywołałem `forward_sequence`, by uzyskać wszystkie `h_t`.
- Obliczyłem wektor score’ów: `scores = tanh(W_a · outputs + b_a)`, a potem softmax wzdłuż wymiaru sekwencji, by uzyskać wagi `att_weights`.
- Ostatecznie skróciłem macierz $(\text{hidden_size} \times \text{seq_len})$ do wektora kontekstowego $(\text{hidden_size} \times 1)$ ważonym sumowaniem.
- **Kluczowe:** dzięki temu nie polegamy tylko na ostatnim stanie LSTM, ale uwzględniam całą historię, ważoną zgodnie z istotnością.

Komórka 3: Tokenizacja i uproszczony BPE

1. `convert_text_to_bytes`

- Konwertuję każdy znak na odpowiadający mu bajt UTF-8. Uzyskuję listę liczb całkowitych, które mogę potem traktować jak „tokeny”.

2. `merge_pair_single`

- Dla podanej pary `(a, b)` skanuję listę tokenów i za każdym razem, gdy napotkam sąsiadującą parę równą `(a, b)`, zastępuję ją nowym tokenem `new_token`.

- Pozwala to łączyć często występujące bigramy w pojedynczy symbol.

3. `iterative_merge`

- W pętli określonej przez `num_iterations` liczę częstotliwość każdej pary, wybieram tę najpopularniejszą i scałem ją raz funkcją `merge_pair_single`.
- Po każdej iteracji zwiększam wartość identyfikatora nowego tokenu.
- **Dlaczego to kluczowe?** Pozwala zredukować długość sekwencji i zmniejszyć słownik bez konieczności ręcznego definiowania reguł, co poprawia wydajność kolejnych warstw modelu.

Komórka 4: Architektury Generatora i Dyskryminatora w PyTorch

1. Generator (`LSTMGeneratorPyTorch`)

- Przyjąłem losowy wektor `z` (`latent_dim`) i przekształciłem go przez warstwę `Linear` → `reshape` → `LSTM` → `Linear` do sekwencji embeddingów.
- To umożliwia modelowi generowanie sekwencji o długości `seq_len`, na którą składa się zarówno tytuł, jak i treść w postaci wektorów.

2. Dyskryminator (`DiscriminatorPyTorch`)

- Wziąłem sekwencję osadzonych wektorów, przepuściłem przez dwukierunkowy LSTM, a następnie uśredniłem (mean pooling) jego wyjścia wzdłuż osi czasowej.
- Przez liniową warstwę i sigmoidę przekształcam tę uśrednioną reprezentację do pojedynczej wartości: prawdopodobieństwa bycia „prawdziwą” sekwencją.
- **Cel:** generator i dyskryminator walczą – generator próbuje oszukać dyskryminator, dyskryminator staje się coraz lepszy w wykrywaniu wygenerowanych próbek.

Komórka 5: Wczytanie i podział danych

1. Wczytałem plik CSV i wyciągnąłem kolumny `text`, `title`, `label`.
2. Oddzieliłem wiadomości `REAL` i `FAKE`.
3. Zrealizowałem **dwustopniowy split**:
 - Połowę próbek `REAL` wrzuciłem do `dfReal`.
 - Pozostałych `REAL` połączyłem z całą grupą `FAKE`, a następnie ponownie pociąłem na połowy, tworząc `dfFake`.
4. Dzięki temu oba `DataFrame`'y mają porównywalne rozmiary, redukując ryzyko dominacji jednej klasy w trakcie treningu.

Komórka 6: Zastosowanie tokenizacji i BPE

- Dla każdej ramki (`dfReal`, `dfFake`):
 1. Zamieniłem kolumny `title` i `text` na listy bajtów.
 2. Przeprowadziłem pięć iteracji łączenia par (funkcja `iterative_merge`).

- Rezultat: każda wiadomość ma teraz dwie listy tokenów, znacznie krótsze od pierwotnego ciągu bajtów, ale zawierające najczęstsze bigramy.

Komórka 7: Warstwa embeddingów

1. Zebrałem wszystkie unikalne tokeny z obu DataFrame'ów, by ustalić `num_embeddings`.
 2. Stworzyłem `nn.Embedding(num_embeddings, embedding_dim=5)`.
 3. Dla każdej sekwencji tokenów (tytułu i tekstu) wygenerowałem 5-wymiarowe embeddingi i zapisałem je jako NumPy.
- **Dlaczego?** Gęste wektorowe reprezentacje poprawiają zdolność LSTM do wychwytywania semantyki danych tekstowych.

Komórka 8: Ekstrakcja stanów ukrytych z LSTM+Attention

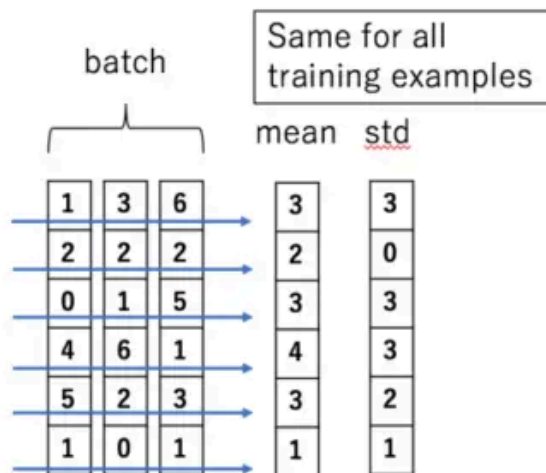
- Korzystając z `LSTMWithAttention`, dla każdej sekwencji embeddingów:
 1. Uzyskałem pełną macierz stanów ukrytych (`hidden_size×seq_len`).
 2. Obliczyłem wektor uwagi i wyciągnąłem kontekstowy wektor `.
 3. Zwróciłem też finalny stan ukryty.
- **Efekt:** każdy tekst i każdy tytuł reprezentuję dwoma wektorami (`hidden_size×1`), które skumulowały informację z całej sekwencji.

Komórka 9: Scalanie i normalizacja

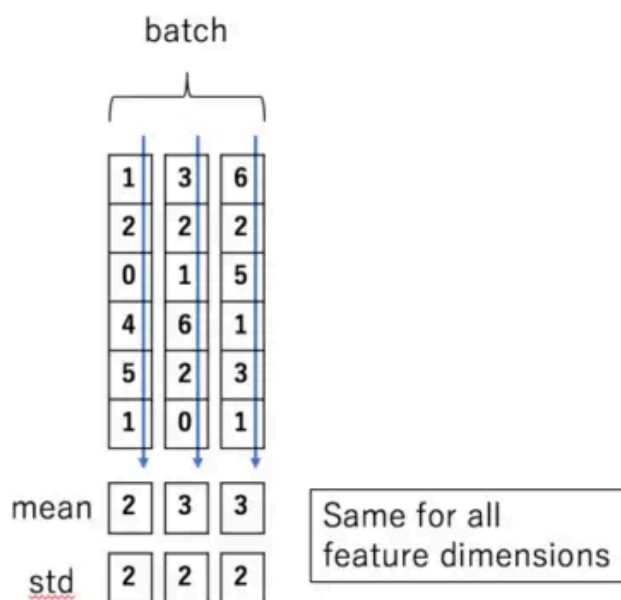
1. Ułożyłem pary wektorów `[stan_tytułu, stan_tekstu]` w macierz (`2×hidden_size`).
2. Przekształciłem je na tensory pytorchowskie i zastosowałem `LayerNorm` wzdłuż wymiaru `hidden_size`, by wyrównać rozkład wartości.

Wizualizacja oraz porównanie do Batch normalization:

Batch Normalization



Layer Normalization



- Normalizacja stabilizuje naukę GAN-a, bo ani generator, ani dyskryminator nie mają „przewagi” wynikającej z różnej skali wejść.

Komórka 10: Przygotowanie DataLoaderów

- Z `real_embeds` oraz `fake_embeds` utworzyłem `TensorDataset` i `DataLoader` z `batch_size=16` i `shuffle=True`.
- **Dlaczego?** PyTorch `DataLoader` zapewnia wydajne podawanie partii danych z losowym przetasowaniem, co jest krytyczne do stabilnego treningu.

Komórka 11: Inicjalizacja modeli, optymalizatorów i funkcji straty

1. Przeniosłem generator i discriminator na GPU (jeśli dostępne).
 2. Ustawiłem optymalizatory `Adam(lr=0.0002)` dla obu sieci.
 3. Wybrałem `BCELoss` jako funkcję mierzącą różnicę między predykcjami dyskryminatora a etykietami.
- **Cel:** przygotować środowisko do treningu naprzemiennego obu sieci.

Komórka 12: Funkcja `train_step`

1. Dyskryminator

- Obliczam stratę na prawdziwych przykładach (`label=1`) i na wygenerowanych bez aktualizacji generatora (`label=0`).

- Suma strat (średnia) to ostateczny `loss_D`, którego gradienty aktualizują tylko parametry dyskriminatora.

2. Generator

- Zamrażam dyskriminator, generuję nowe próbki z losowego `z` i nakładam etykietę 1 (generator „oszukuje”).
- `loss_G` to `BCELoss(discriminator(gen(z)), 1)`, który aktualizuje parametry tylko generatora.
- **Dlaczego?** To klasyczny schemat treningu GAN: dyskriminator i generator uczą się na przemian, co prowadzi do wzajemnej konkurencji i coraz lepszych generowanych próbek.

Komórka 13: Pętla treningowa

- Przez **2100 epok**: dla każdej partii danych z prawdziwych przykładów wywołuję `train_step`, agreguję średnie straty i co 10 epok wypisuję postęp.
- **Wynik**: dwie listy `losses_D` i `losses_G`, które potem wizualizowałem (wcześniej pokazywałem wykres).

Komórka 16: Detekcja anomalii i wizualizacja rozkładu

1. Procedura detekcji

- Dla każdej próbki `sample` optymalizuję wektor `z` (rozpoczynając od losowego), minimalizując błąd rekonstrukcji między `generator(z)` a `sample`.
- Po kilkudziesięciu krokach (100 iteracji, `lr=0.01`) otrzymuję najlepsze `z*`.

2. Wskaźnik anomalii

- `residual_loss = MSE(generator(z*), sample)`
- `discriminator_score = D(sample)`
- `anomaly_score = (1 - discriminator_score) + λ · residual_loss`

3. Wizualizacja

- Rysuję histogramy rozkładów `real_scores` i `fake_scores`.
- Ustawiam próg na 80-percentylu prawdziwych i liczę, ile dew punktów przekracza go w każdej klasie.
- **Dlaczego kluczowe?** AnoGAN wykorzystuje jednocześnie zdolność generatora do rekonstrukcji i dyskriminatora do oceny autentyczności, co daje mocną miarę odchylenia od „normalnego” rozkładu danych.

Optymalizacja przy pomocy "Adam" pozwala modelowi ominąć problem wpadnięcia w minimum lokalne. Adam to wydajny algorytm używany do trenowania modeli uczenia maszynowego, szczególnie w uczeniu głębokim. Łączy w sobie zalety dwóch innych metod, Momentum i RMSprop, aby adaptacyjnie dostosować szybkość uczenia się dla każdego parametru, co prowadzi do szybszej konwergencji i poprawy wydajności.

Minimum lokalne to punkt na powierzchni funkcji, w którym wartość funkcji jest niższa niż we wszystkich „sąsiednich” punktach, ale niekoniecznie jest to najniższa możliwa wartość w całej przestrzeni.

definicje:

Minimum lokalne = punkt niemalejący wokół siebie, ale wyższy od globalnego dna.

Minimum globalne = najniższy punkt w całej przestrzeni.

W kontekście trenowania sieci: unikamy lokalnych minimów, by model osiągnął jak najniższą stratę.

Dlaczego to ma znaczenie w uczeniu maszynowym?

Podczas trenowania sieci neuronowej optymalizator (np. właśnie Adam) **minimalizuje funkcję straty** – szuka najniższego punktu. Jeśli wpadnie w **minimum lokalne**, może „utknąć” i nie znaleźć lepszego globalnego rozwiązania. To oznacza, że model może nie osiągnąć możliwie najlepszej jakości.

1. LSTM z mechanizmem Attention

Problem z czystym LSTM polega na tym, że choć sam potrafi wyciągnąć sekwencyjny kontekst, to przy długich tekstach cała informacja często kumuluje się w ostatnim stanie ukrytym. To sprawia, że wcześniejsze ważne fragmenty mogą zostać „zapomniane” lub przyćmione przez późniejsze elementy sekwencji.

Moim rozwiązaniem jest mechanizm uwagi. Pozwalam modelowi „zwrócić wzrok” na wszystkie kroki sekwencji równomiernie, przyznając im wagi proporcjonalne do ich istotności. Dzięki temu osiągam lepszą reprezentację, ponieważ każda pozycja w tekście ma szansę wpłynąć na ostateczny wektor reprezentujący całą sekwencję. Co więcej, zyskuję wyjaśnialność – mogę sprawdzić, które fragmenty tekstu model uznał za kluczowe, patrząc na rozkład wag attention.

W moim projekcie to właśnie uwaga pozwala mi „wyciągnąć” te części tytułu i treści, które najbardziej różnicują prawdziwe wiadomości od fałszywych. Bez tej warstwy ryzyko utraty sygnału w długich wiadomościach byłoby znacznie większe.

2. Byte-Pair Encoding (BPE)

Problem, który napotykam, dotyczy słów i znaków. Pełny zestaw znaków UTF-8 (lub słów) może być ogromny, co prowadzi do bardzo dużych „słowników” (miliony tokenów). To spowalnia zarówno embedding, jak i kolejne warstwy sieci.

Moim prostym rozwiązaniem jest BPE. Iteracyjnie łączę najczęstsze pary bajtów, co redukuje długość sekwencji i liczbę unikalnych tokenów, pozostawiając w słowniku tylko te n-gramy, które rzeczywiście się powtarzają. Przynosi to korzyści takie jak mniejszy słownik, co oznacza szybsze uczenie warstwy embedding i mniejszą potrzebę pamięci. Jednocześnie

zachowuję kluczowe n-gramy – najczęstsze bigramy (np. „th”, „in”, „ed”) stają się pojedynczymi tokenami, co zachowuje sens semantyczny, lecz oszczędza miejsce.

W moim projekcie BPE ułatwia modelowi wychwytywanie wzorców tekstowych charakterystycznych dla fake news (np. często powtarzających się fraz) bez przeciążania sieci zbyt długimi wektorami.

3. Embeddingi

Zauważyłem, że Generator i Dyskryminator działają na wartościach ciągłych. Bezpośrednie użycie tokenów (liczb całkowitych) nie miałoby sensu w warstwach LSTM czy liniowych.

Moim rozwiązaniem jest warstwa Embedding. Każdy token (bajt, bigram) jest mapowany na gęsty wektor n-wymiarowy. Te wektory uczą się odwzorowywać podobieństwa (np. bigramy o zbliżonym znaczeniu znajdują się blisko w przestrzeni) i pozwalają LSTM operować na wartościach rzeczywistych, co jest niezbędne do gradientowego trenowania.

W moim projekcie embeddingi stanowią „wejście wysokiego poziomu” dla LSTM+Attention i dla Generators/Dyskryminatora w GAN, pozwalając modelom uczyć się semantyki tekstu, a nie tylko sekwencji surowych liczb.

4. Scalanie stanów tytułu i treści oraz normalizacja (LayerNorm)

Każda wiadomość, z którą pracuję, ma dwa źródła informacji: tytuł i treść. Moim podejściem jest wspólne zestawienie ich stanów ukrytych w jedną strukturę, co pozwala GAN-owi ocenić cały kontekst, a nie tylko jeden aspekt danych.

Jednak napotykam problem z różnymi rozkładami wartości z LSTM, które w różnych skalach mogą wprowadzać niestabilność podczas treningu GAN-a. Generatorowi i dyskryminatorowi trudniej się „dogadać”, jeśli ich wejścia krążą wokół różnych średnich i odchyień.

Moim rozwiązaniem jest LayerNorm. Wyrównuję średnią i wariancję wektorów wzdłuż wymiaru ukrytego, co stabilizuje gradienty w GAN-ie, umożliwia szybsze zbieganie się modelu i zapobiega „przeskakiwaniu” jednych wejść nad drugimi.

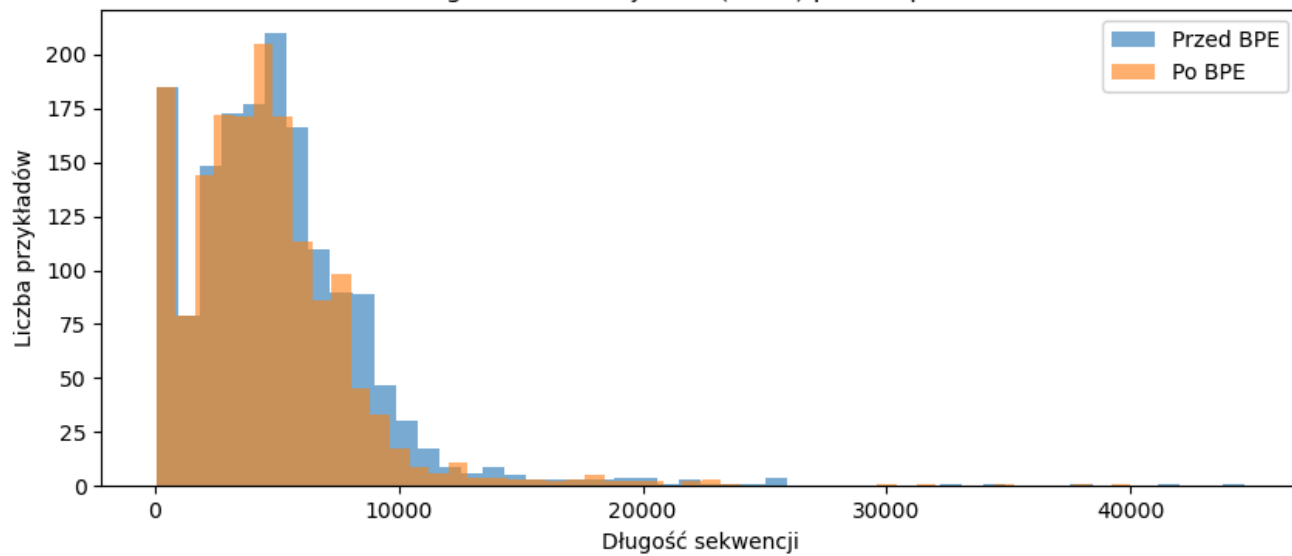
W moim projekcie dzięki znormalizowanym, połączonym wektorom model łatwiej identyfikuje subtelne różnice między prawdziwymi a fałszywymi wiadomościami, a trening jest bardziej przewidywalny.

Wizualizacja przebiegu uczenia oraz opis

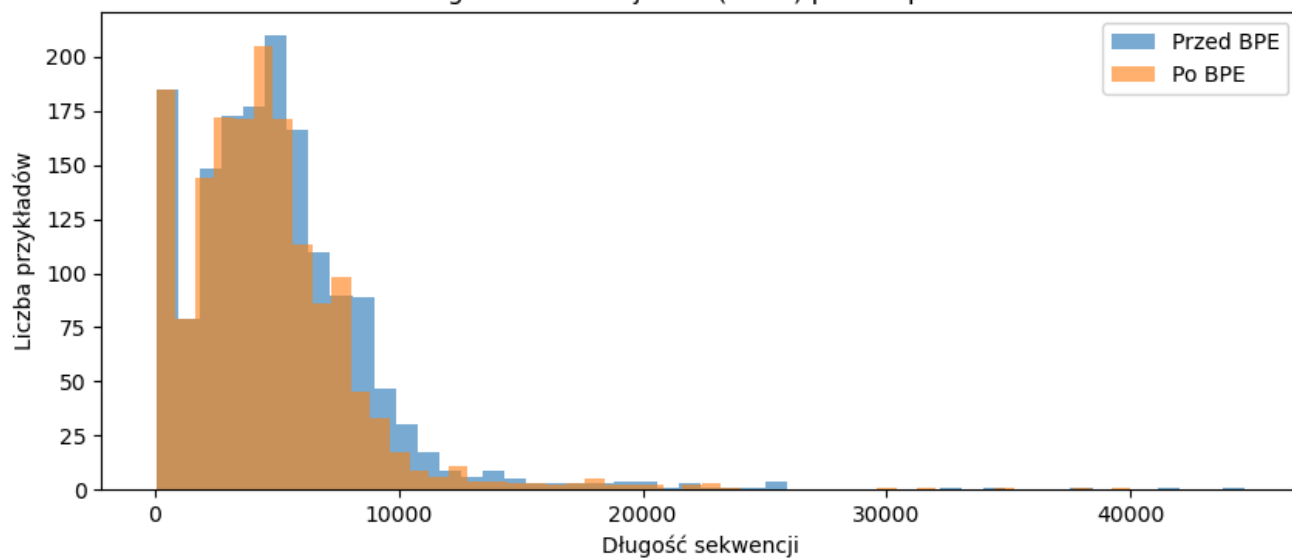
Algorytm BPE jest dobry, ale nie można przesadzić z "mocą" skracania, ponieważ później nadmierna liczba iteracji prowadzi do tworzenia długich, rzadkich tokenów, które rozmywają semantykę i zwiększają koszt obliczeniowy. Kluczem jest więc znalezienie kompromisu między kompresją a zdolnością modelu do generalizacji.

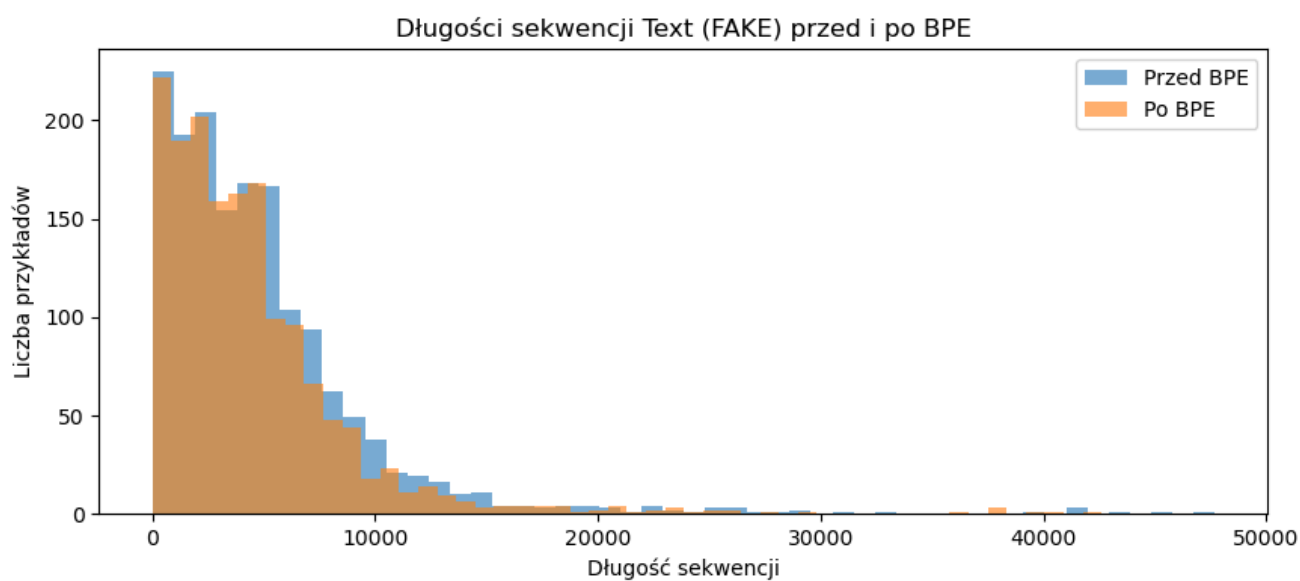
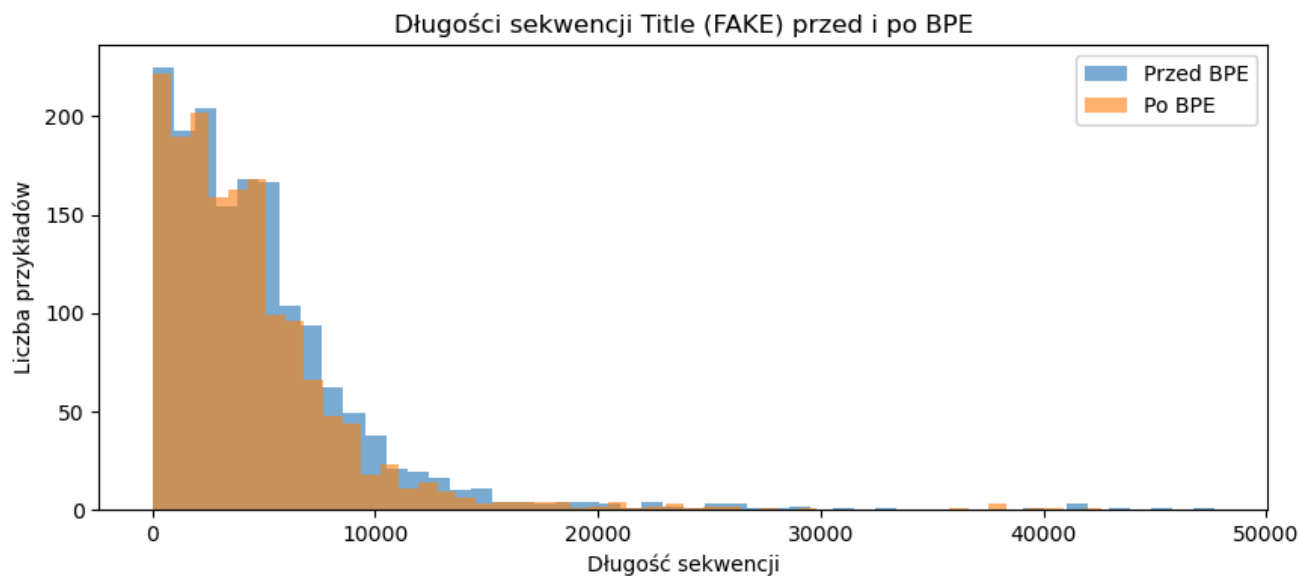
Wizualizacja algorytmu BPE dla różnych danych ze zbioru:

Długości sekwencji Title (REAL) przed i po BPE

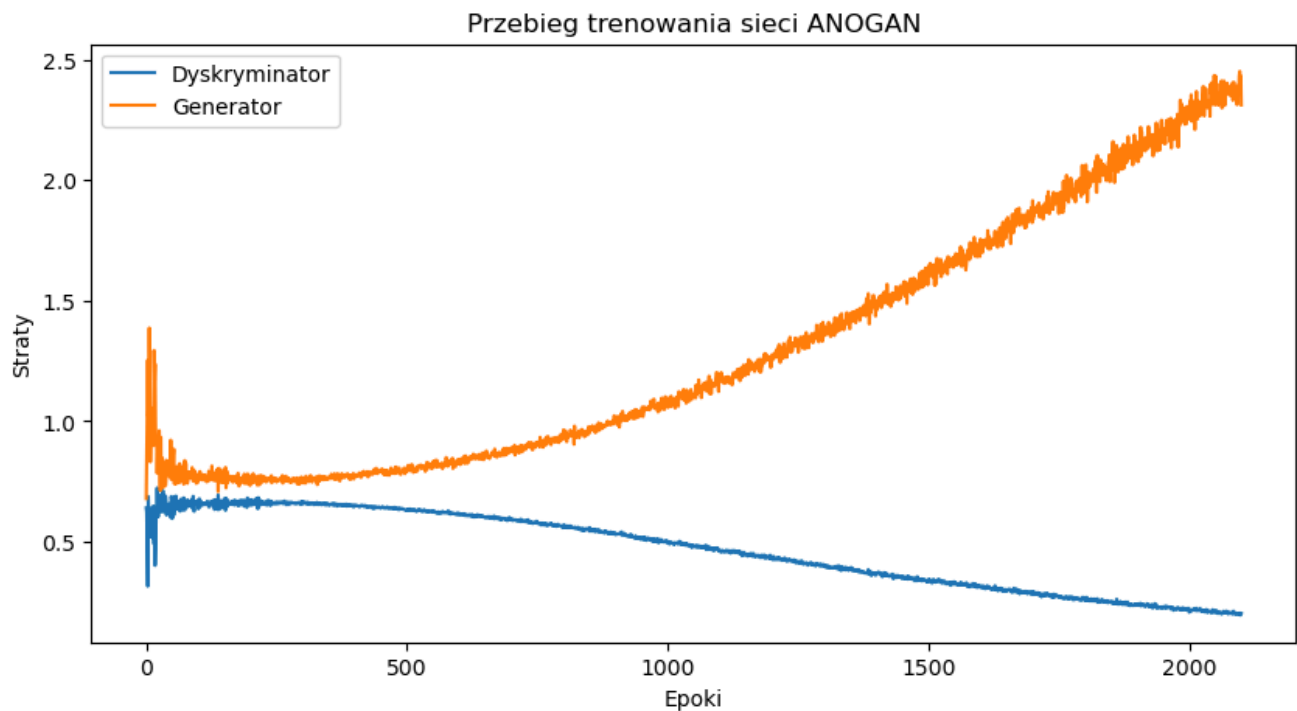


Długości sekwencji Text (REAL) przed i po BPE





Przebieg uczenia się modeli GAN rywalizujących ze sobą



Ten wykres przedstawia **przebieg trenowania sieci ANOGAN (anomaly detection GAN)**, pokazując, jak zmieniają się **straty (loss)** dwóch komponentów sieci GAN — **generatora** i **dyskryminatora** — w czasie (w funkcji epok).

Co pokazuje wykres:

Oś X:

- Liczba **epok** treningowych (czyli ile razy dane zostały przetworzone przez model).

Oś Y:

- Wartość **straty (loss)** – im niższa, tym lepiej model się dopasował do danych (przy założeniu odpowiedniej równowagi).

Linie:

- **Niebieska (dyskryminator):** Początkowo wysoka, a potem **stopniowo maleje**.
- **Pomarańczowa (generator):** Na początku spada, ale później **systematycznie rośnie**.

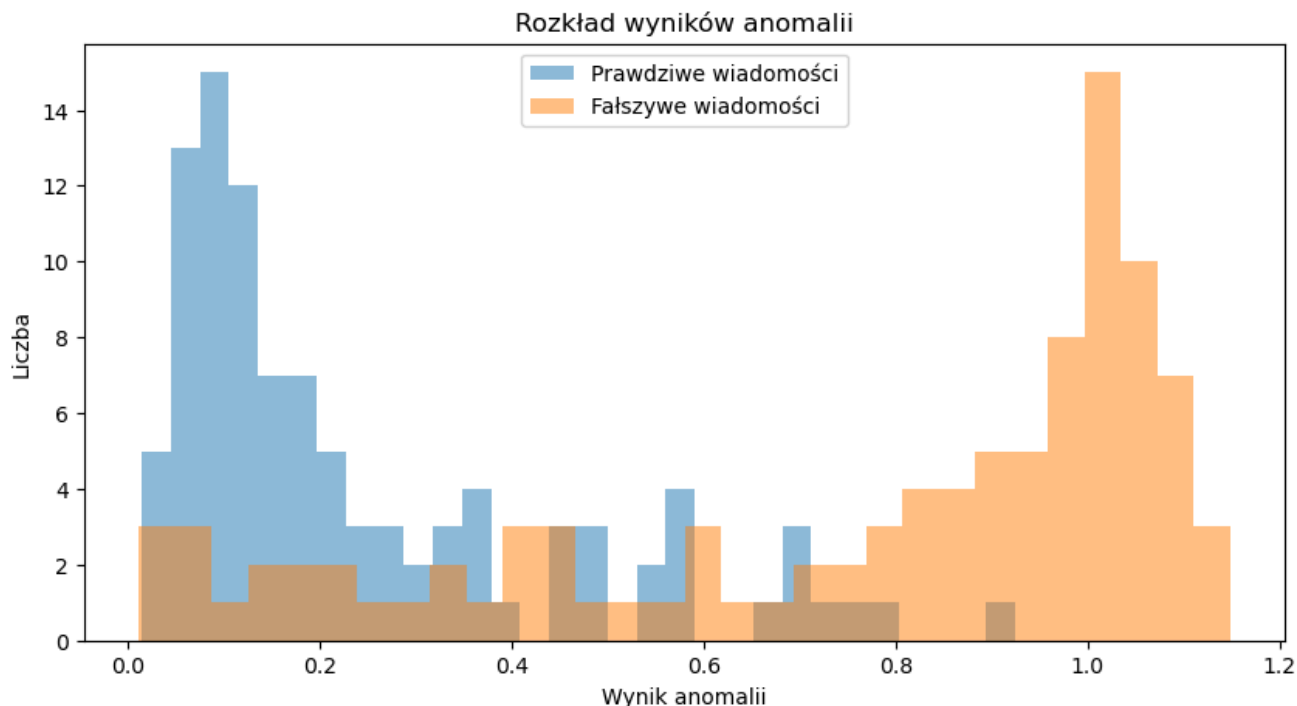
Interpretacja treningu ANOGAN:

1. Na początku treningu:

- Straty obu sieci są dość wysokie i niestabilne.
- Generator szybko się uczy i poprawia swoje wyniki – jego strata maleje.
- Dyskryminator również się poprawia i dostosowuje do rozpoznawania wygenerowanych próbek.

2. Po pewnym czasie:

- **Dyskryminator coraz lepiej odróżnia dane rzeczywiste od fałszywych** – jego strata spada stabilnie.
- **Generator zaczyna tracić „walkę” z dyskryminatorem**, co powoduje, że jego strata rośnie – generowane dane są coraz łatwiejsze do wykrycia jako fałszywe.



Co oznacza wykres:

- **Oś X:** "Wynik anomalii" – prawdopodobnie wartość od 0 (normalna wiadomość) do 1 (anomalna wiadomość).
- **Oś Y:** Liczba wiadomości (czyli ile wiadomości miało dany wynik).
- **Niebieski słupek:** Prawdziwe wiadomości
- **Pomarańczowy słupek:** Fałszywe wiadomości

Co z niego wynika:

- **Prawdziwe wiadomości** mają **niskie wyniki anomalii** (głównie 0.0–0.4), co oznacza, że model uznał je za "normalne".
- **Fałszywe wiadomości** mają **wysokie wyniki anomalii** (głównie 0.7–1.1), co oznacza, że model uznał je za "anomalne".

Wniosek:

Model dobrze oddziela prawdziwe od fałszywych wiadomości na podstawie wyniku anomalii. Histogram pokazuje, że **rozzrūt wyników dla obu klas jest wyraźnie rozdzielony**, co sugeruje skuteczność detekcji anomalii.